BIG DATA · UDEM

Big Data e Ingeniería de Datos

Clase 2 · Cómo se lee big data y SQL

Posgrado · UdeM · 4 nov 2026

239.5 M

viajes FHVHV de 2024. La clase 1 consultó gold, ya agregado: 15 909 filas. Leer el crudo completo exige manejar esta cifra.

NYC TLC · Query Profile, clase 1

PREGUNTA DE LA CLASE

¿Cómo lee un motor analítico 600 M de filas en menos de un segundo, en caliente?

LINEITEM de TPC-H, en un warehouse X-Small

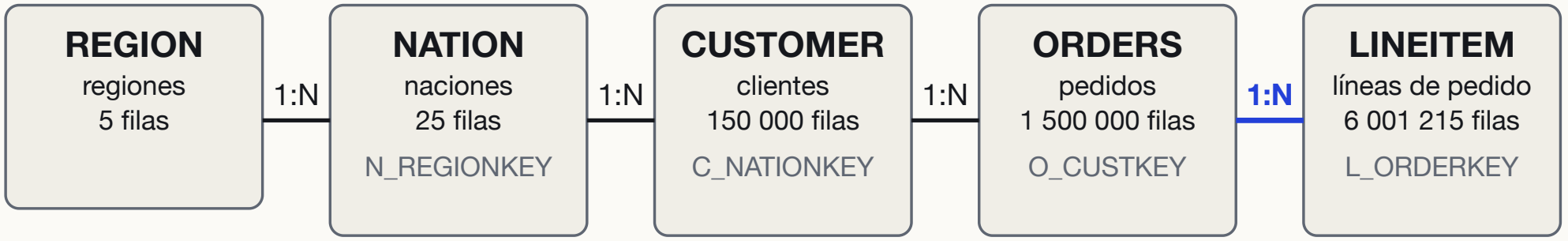
Sumar el importe de 600 M de líneas de pedido tarda 0.6 s en caliente y 2.5 s en frío en X-Small

```
SELECT SUM(L_EXTENDEDPRICE) AS IMPORTE
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF100.LINEITEM;
```

0.63 y 0.67 s en caliente (datos ya en caché), 2.50 s en frío. TableScan (el operador que lee la tabla): 1.83 GB.

TPC-H es la base de ejemplo de un mayorista: clientes, pedidos, líneas, naciones y regiones

TPC-H simula un mayorista. Los labs usan SF1; las demos de escala, SF100 y SF1000.



Filas de LINEITEM por factor de escala (SF)

SF1 · 6 M	SF10 · 60 M	**SF100 · 600 M**	SF1000 · 6 000 M

PART y SUPPLIER no se usan; PARTSUPP, solo en la slide 53. SF1 pesa 165 MB en LINEITEM.

LINEITEM: 6 M de filas en SF1; 600 M en SF100; 6 000 M en SF1000.

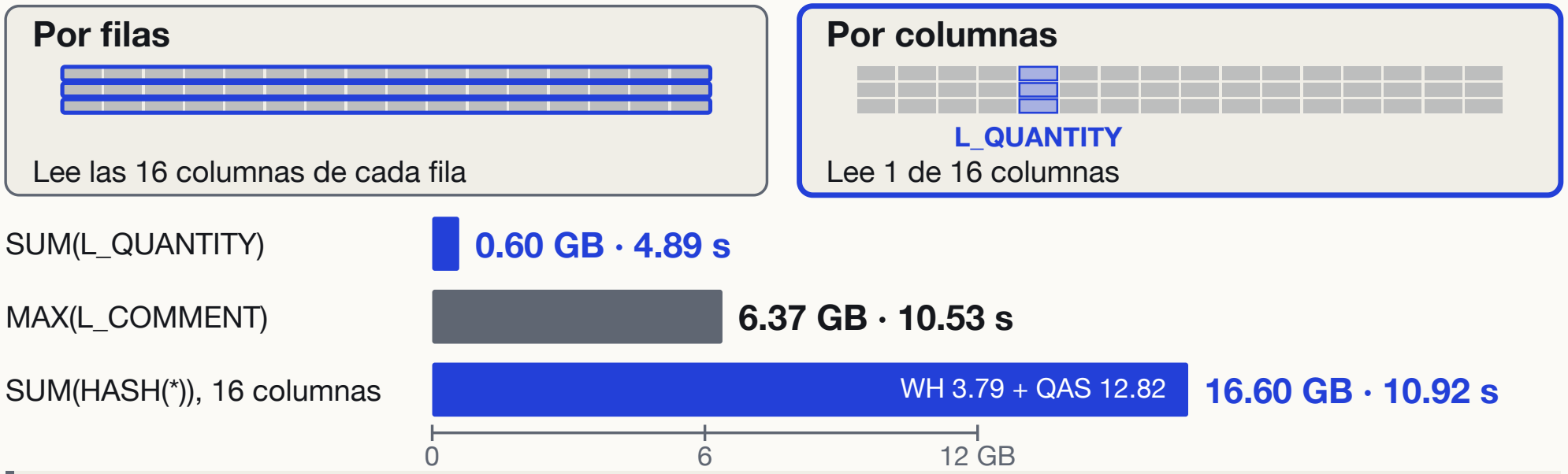
Leer big data plantea ocho problemas; cada uno tiene síntoma medido, idea de ingeniería y respuesta de Snowflake

Problema	Síntoma con dato	Idea de ingeniería	Snowflake
1 **Columnar**	0.60 GB frente a 6.37 GB	Leer solo columnas	**Micro-particiones**
2 **Pruning**	134 de 10 336 particiones	Omitir por rango	**Rango por columna**
3 **Escalar**	10× filas, unas 9× tiempo	Separar cómputo y datos	**Warehouses por talla**
4 **Aislar cargas**	Carga de 6.0 s con consulta de 0.79 s	Cómputo por carga	**Un warehouse por carga**
5 **Costo y elasticidad**	Medium: 1.45× más rápido, 2.8× el costo	Pagar lo encendido	**Auto-suspend y caché**
6 **Datos crudos**	15 de 24 columnas usadas	Esquema flexible	VARIANT
7 **Transformación**	Filtros, 2 uniones, aplanado	Código con pruebas	SQL; pipeline aparte
8 **Formatos abiertos**	5.4 GB de Parquet en S3	Archivos abiertos	Tablas Iceberg

Warehouse: cómputo de Snowflake. X-Small: talla menor, 1 crédito (unidad de cobro) por hora

La lectura depende de qué columnas pide la consulta: 0.6 GB una numérica, 6.4 GB un texto, 16.6 GB la fila completa

Las columnas se guardan por separado: la consulta lee solo las que nombra.

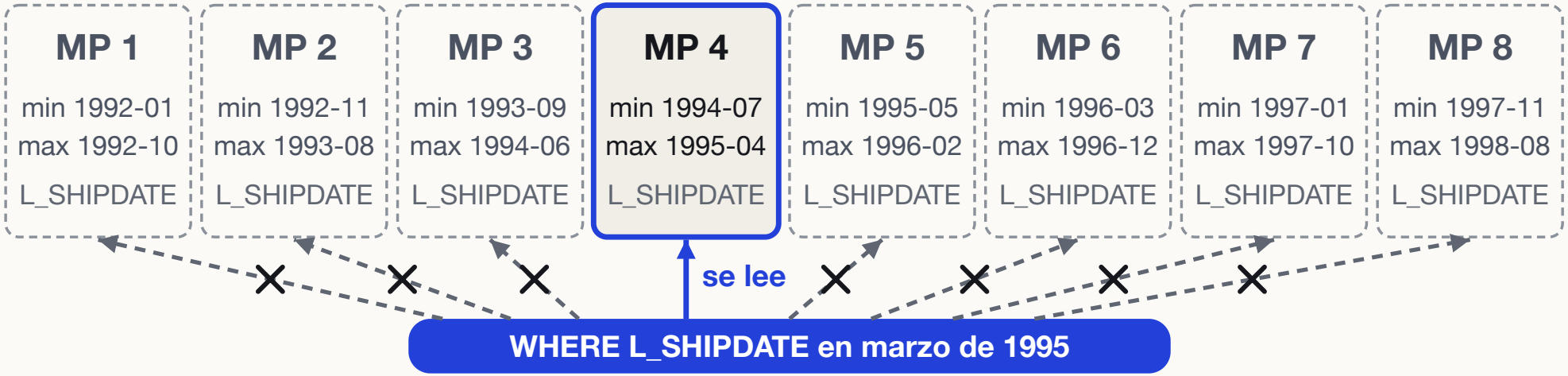


Leer las 16 columnas toca todas las columnas. TPCH_SF100, X-Small, caché fría.

Mismo LINEITEM en SF100; 16 columnas guardadas por separado.

Cada micro-partición guarda el rango de valores de sus columnas, y el filtro descarta las que no pueden contener filas

El orden de carga agrupa las fechas: cada micro-partición guarda el rango de cada columna y el filtro salta las que no encajan.



134 de 10 336 micro-particiones · **19.2 GB → 0.25 GB** · **6.0 s → 2.0 s**

SF1000, X-Small, un mes de L_SHIPDATE; sin filtro, media de cuatro ejecuciones

Límite: omite solo si la columna está agrupada. Profundidad media: 1.41 en L_SHIPDATE; 10 336 en L_QUANTITY (todas se solapan). Rangos del diagrama: ilustrativos.

Micro-partición: bloque de 50–500 MB; SF1000, un mes: 134 de 10 336

Filtrar por un mes baja el tiempo de 6.0 s a 2.0 s y la lectura de 19.2 GB a 0.25 GB

```
SELECT SUM(L_EXTENDEDPRICE)
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1000.LINEITEM
WHERE L_SHIPDATE >= '1995-03-01' AND L_SHIPDATE < '1995-04-01';
```

Con la línea 3: 134 de 10 336 micro-particiones, en X-Small. Los bytes bajan más que el tiempo.

0.06 s

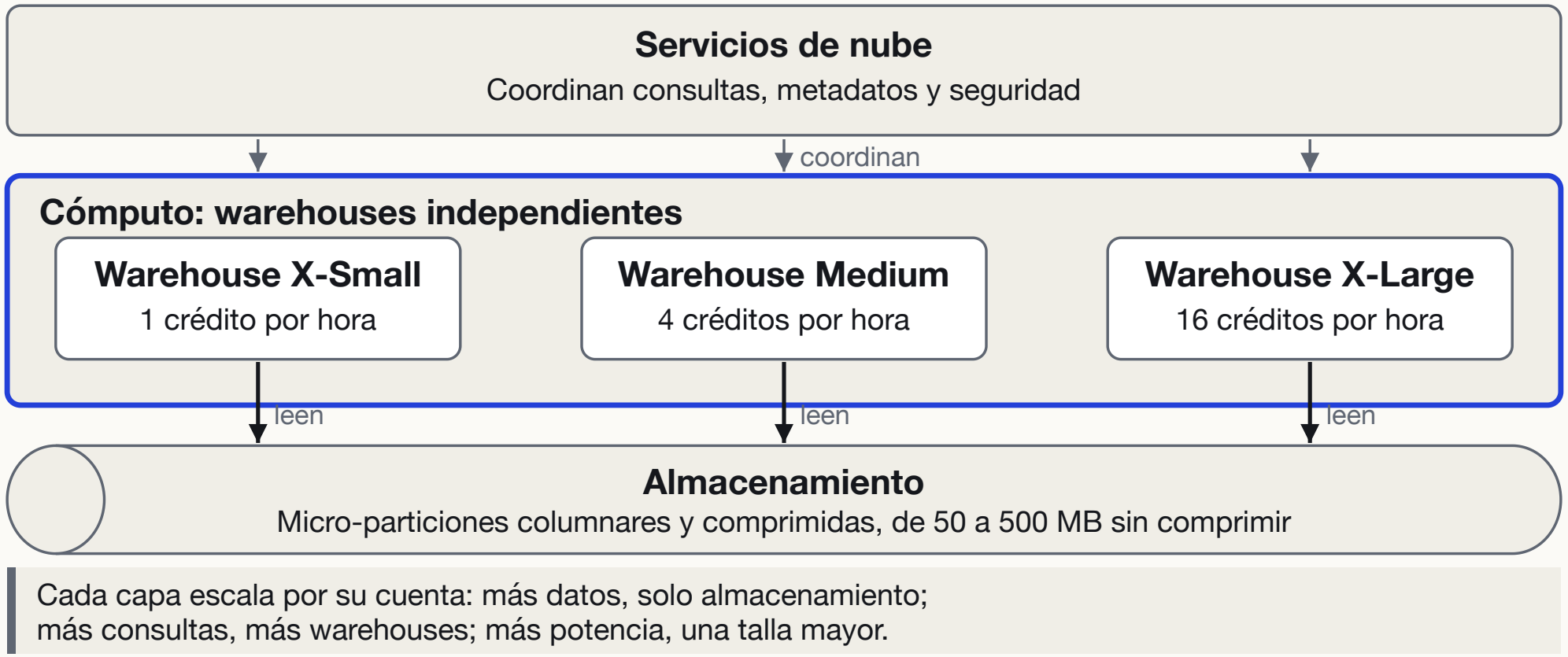
COUNT(*) sobre 600 M de filas lee 0 bytes. Leer L_QUANTITY (SUM) sí abre la columna: 0.60 GB.

Medido · TPCH_SF100 · X-Small

Con 10 veces más filas (SF100 → SF1000), la misma consulta tarda unas 9 veces más en X-Small

Escala	Filas	Tiempo medio	GB totales
SF100	600 M	0.65 s	1.83 GB
SF1000	6 000 M	6.0 s	19.2 GB

Snowflake separa almacenamiento, cómputo y servicios de nube, y cada capa escala por su cuenta



> Cada capa escala por su cuenta: más datos, solo almacenamiento; más consultas, más warehouses; más potencia, una talla mayor.

Un warehouse va de X-Small (1 crédito por hora) a 6X-Large (512).

Varios warehouses leen el mismo almacenamiento sin competir por cómputo: una carga pesada no frena a las demás

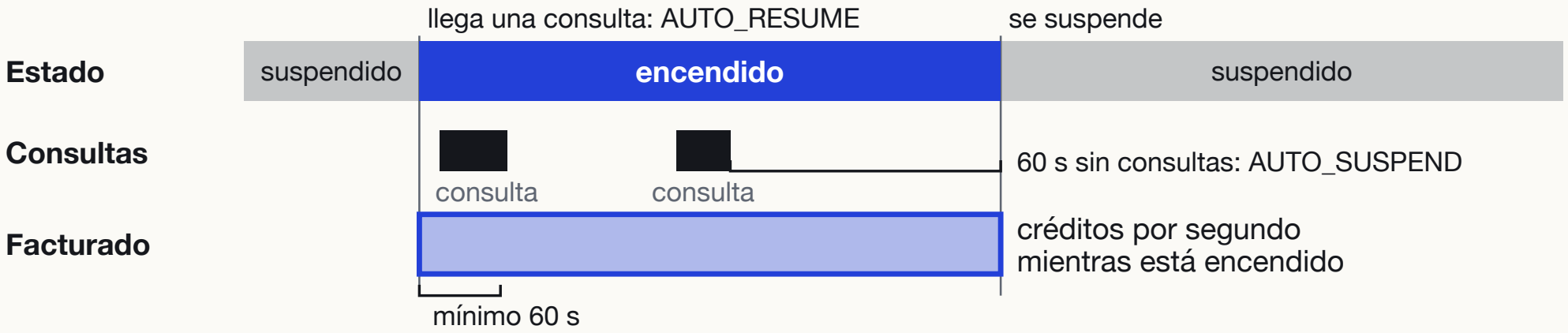


Cada warehouse tiene su propio tiempo y no afecta a los demás; cada uno encendido factura sus créditos.

Cada warehouse encendido factura sus propios créditos.

El warehouse cobra mientras está encendido: auto-suspend lo apaga y cada arranque cobra al menos 60 segundos

Se factura el tiempo encendido, por segundo, con un mínimo de 60 s en cada arranque.



0.0167 créditos
un arranque mínimo, X-Small

0.0667 créditos
un arranque mínimo, Medium

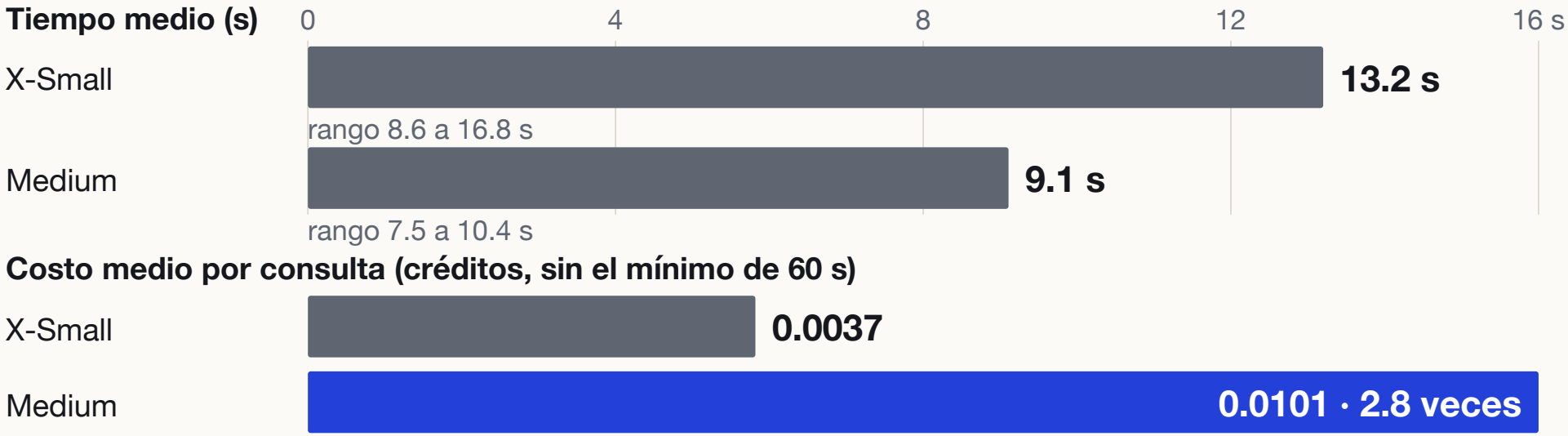
0.0207 créditos
19 consultas, 74.6 s, X-Small

Costo = talla × tiempo encendido, no consultas ejecutadas. Apagar rápido reduce el tiempo ocioso.

X-Small: 1 crédito por hora; el mínimo de un arranque son 0.0167 créditos.

Medium terminó 1.45 veces más rápido que X-Small y costó 2.8 veces más por consulta

Misma consulta y mismo volumen: 9 144 de 10 336 micro-particiones en las ejecuciones completas; tres ejecuciones por talla.



Medium fue 1.45 veces más rápido y costó 2.8 veces más. Con n = 3, la diferencia de tiempo es indicativa.

Media de 3 ejecuciones: 13.2 contra 9.1 s. Spilling: datos que exceden la memoria van a disco.

La misma consulta repetida sale del caché de resultados: 2.14 s la primera vez, 0.05 s la segunda y 0 bytes

```
ALTER SESSION SET USE_CACHED_RESULT = TRUE;

SELECT L_SHIPMODE, COUNT(*), SUM(L_EXTENDEDPRICE)
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF100.LINEITEM
GROUP BY 1;  -- ejecútala dos veces
```

Caché de resultados (24 h): texto idéntico, datos sin cambios, rol con privilegios. Caché local (SSD del warehouse): se pierde al suspender.

VARIANT guarda el JSON del clima sin esquema fijo, pero el crudo queda en el almacenamiento y el formato de Snowflake

JSON de Open-Meteo

```
{"daily": {
  "time": ["2024-01-01", …],
  "precipitation_sum": [0.0, …]
}}
```

cargar

Columna VARIANT

consultar por ruta

fecha	lluvia_mm
2024-01-01	0.0
…	…

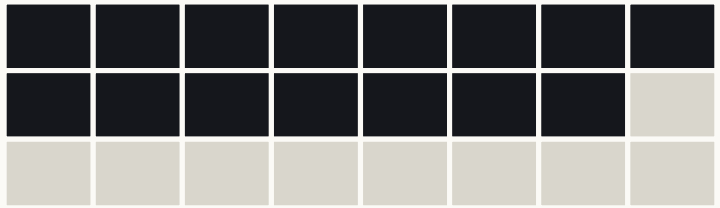
Tabla de viajes

Almacenamiento de Snowflake

Formato propio de Snowflake

sin salida abierta

Otros motores sobre archivos



15 de 24 columnas usadas

> Resuelve: esquema flexible para JSON.
> No resuelve: el crudo en archivos abiertos de cualquier motor.

FHVHV: 24 columnas, 15 usadas; el clima llega anidado en daily.

Snowflake puede ejecutar filtros, 2 uniones y aplanado de silver; el curso los escribe como código con pruebas

Snowflake

– Uniones y filtros en SQL, JSON en VARIANT, PySpark con Snowpark Connect

Curso

– Pruebas automáticas, reproceso por mes, orden y reintentos

Las tablas nativas usan formato propio; las Iceberg guardan Parquet, administradas por Snowflake o en el bucket del cliente



> En el curso: bronze es Parquet en S3 (clase 4); gold se sirve en una tabla de Snowflake.
> Sin Fail-safe en el bucket del cliente.

Sin Fail-safe en el bucket del cliente.

Snowflake resuelve cinco problemas de lectura; el diseño del curso, los otros tres

Motor

– Columnas

– Pruning

– Escala

– Aislamiento

– Pago por segundo

Diseño

– Lake en S3

– Spark y Airflow

– Parquet abierto

BigQuery on-demand cobra por bytes leídos; los otros tres, por tiempo encendido

Motor	Cobro	Aísla	Pausa
Snowflake	Créditos/segundo	Warehouses	Auto-suspend
Redshift	Nodo-hora o RPU-hora	Data sharing	Pausa manual (provisioned)
BigQuery	Bytes o slots	Reservas	Sin clúster
Databricks SQL	DBU	—	Auto Stop

Los cuatro omiten datos con metadatos; Databricks expone Delta como Iceberg con UniForm

Motor	Omite datos	Formato abierto
Snowflake	Rango por micro-partición	Iceberg
Redshift	Zone maps (mín/máx por bloque)	Spectrum, Iceberg
BigQuery	Particiones, clustering	Iceberg
Databricks SQL	Min/máx por archivo	Delta, UniForm

02

PARADA AWS

¿Qué cuotas de tu cuenta de AWS deciden si el bootstrap y el Code Editor arrancan?

Tu color del semáforo de la clase 1 (formulario) decide qué haces en esta parada

Color	Estado	Ahora
Rojo	En verificación	Empieza Lab 1
Amarillo	Verificada	Pide cuotas
Verde claro	Bootstrap hecho	Empieza Lab 1
Verde	Pasos 5 a 12	Retos ★★

Cuatro cuotas de us-east-2 deciden si el bootstrap y el Code Editor arrancan

Servicio	Cuota	Mínimo
SageMaker	Studio user profiles	1
SageMaker	CodeEditor Apps en ml.t3.large	1
SageMaker	Total domains	1
EC2	Running On-Demand Standard	2 vCPU

Revisa tus cuotas con CloudShell en us-east-2 (paso 2.2)

Consola AWS

1. Abre CloudShell (ícono >_) en us-east-2.
2. Pega el comando del paso 2.2 de docs/guia.md.
3. Cada valor: mayor o igual al mínimo.
4. Si dice «verification is in progress», espera el correo.

Pide las cuotas que falten (paso 2.3)

Consola AWS

1. Service Quotas > servicio > cuota > Request increase.
2. Escribe el valor por defecto (Code Editor: 6).
3. Estado en Quota request history.
4. Si es un caso, responde con tu caso de uso.

03

LAB 1

¿Cómo se le pregunta a una tabla? Tus primeras consultas SQL sobre TPC-H

Copia lab1.sql desde tu fork con Copy raw file

GitHub

1. Abre tu fork > curso > clase02 > lab1.sql.
2. Pulsa Copy raw file: dos cuadritos, no la flecha.
3. Si no ves el archivo: Sync fork > Update branch.

Pega el archivo en Snowsight y corre el Paso 0

Snowsight

1. Projects > Worksheets > + SQL Worksheet.
2. Pega todo el archivo.
3. Selecciona de USE ROLE a USE SCHEMA y pulsa ▶.
4. Corre cada ejercicio con Cmd o Ctrl + Enter.

La verificación responde OK, REVISAR o PENDIENTE por ejercicio; la meta es RESUMEN con 6 de 6

Resultado	Significa	Qué haces
OK	Mismas filas y valores	Sigue
REVISAR	Existe, no coincide	Lee la PISTA
PENDIENTE	Marcador sin resolver	Escribe tu SELECT

Cada ejercicio es una vista EJn: escribes tu SELECT entre las dos marcas, sin punto y coma

```
CREATE OR REPLACE VIEW EJ1 AS
-- ▼▼ BORRA la línea de abajo y escribe aquí tu SELECT ▼▼
SELECT NULL AS PENDIENTE
-- ▲▲ hasta aquí ▲▲
;
SELECT * FROM EJ1;
```

Una vista guarda la consulta con un nombre fijo. La verificación abre EJ1 a EJ10 y compara su resultado.

Resuelve los seis ★, lee el Query Profile y verifica

Snowsight

1. Resuelve EJ1 a EJ6 (★).
2. EJ6: abre Query Profile (el plan ejecutado); anota las filas de TableScan.
3. Con tiempo: ★★ y ★★★.
4. Otra hoja: lab1_verificacion.sql, Run All. Meta: 6 de 6.

DESCANSO

Vuelves en 20 minutos: GROUP BY, HAVING y el orden lógico de una consulta

04

AGREGACIÓN

¿Cómo se convierten 600 M de líneas en una fila por grupo y en qué orden evalúa el motor la consulta?

GROUP BY reduce 600 M de líneas a 7 filas, una por modo de envío

```
SELECT L_SHIPMODE, COUNT(*), SUM(L_EXTENDEDPRICE)
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF100.LINEITEM
GROUP BY 1;
```

GROUP BY 1 agrupa por la primera columna del SELECT. Cada fila de salida resume un grupo: COUNT(*) cuenta sus líneas y SUM suma su importe.

WHERE filtra filas antes de agrupar y HAVING filtra grupos después de agregar

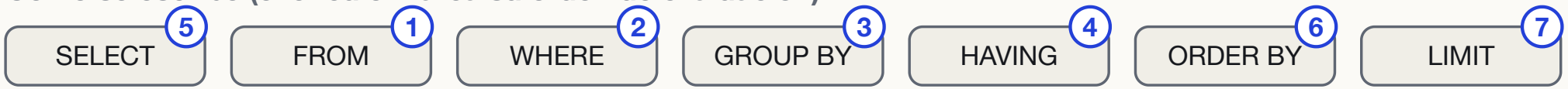
```sql
SELECT O_CUSTKEY, COUNT(*) AS PEDIDOS
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
WHERE O_ORDERDATE >= '1995-01-01' AND O_ORDERDATE < '1996-01-01'
GROUP BY O_CUSTKEY
HAVING COUNT(*) >= 8
ORDER BY PEDIDOS DESC, O_CUSTKEY
LIMIT 5;
```

WHERE deja 228 637 pedidos de 1995; HAVING deja los 622 clientes con 8 o más. Resultado: 5 filas, de 12 a 11 pedidos.

El motor evalúa FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY y LIMIT, en ese orden lógico

Se escribe empezando por SELECT; se evalúa empezando por FROM.

Cómo se escribe (el círculo marca su orden de evaluación)



Cómo se evalúa



Filas que quedan (SF1, consulta de la slide 37)

1	2	3	4	5	6	7
1 500 000 pedidos	**228 637** de 1995	**86 573** grupos	**622** con 8 o más	**622** filas, 2 columnas	**622** ordenadas	**5** filas

WHERE no ve agregaciones: aún no existen. ORDER BY sí ve los alias del SELECT.
Es un orden lógico, no físico.

Orden lógico, no físico: el optimizador puede reordenar si el resultado no cambia.

WHERE filtra filas y no admite agregaciones; HAVING filtra grupos y sí las admite

Cláusula	WHERE	HAVING
Se evalúa	Antes de GROUP BY	Después
Filtra	Filas	Grupos
Agregaciones	No	Sí

Una columna del SELECT debe estar en GROUP BY o dentro de una agregación; si no, la consulta falla

```
SELECT C_MKTSEGMENT, C_NAME, COUNT(*)
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER
GROUP BY C_MKTSEGMENT;
```

Snowflake responde: 'CUSTOMER.C_NAME' in select clause is neither an aggregate nor in the group by clause. Un segmento agrupa miles de nombres.

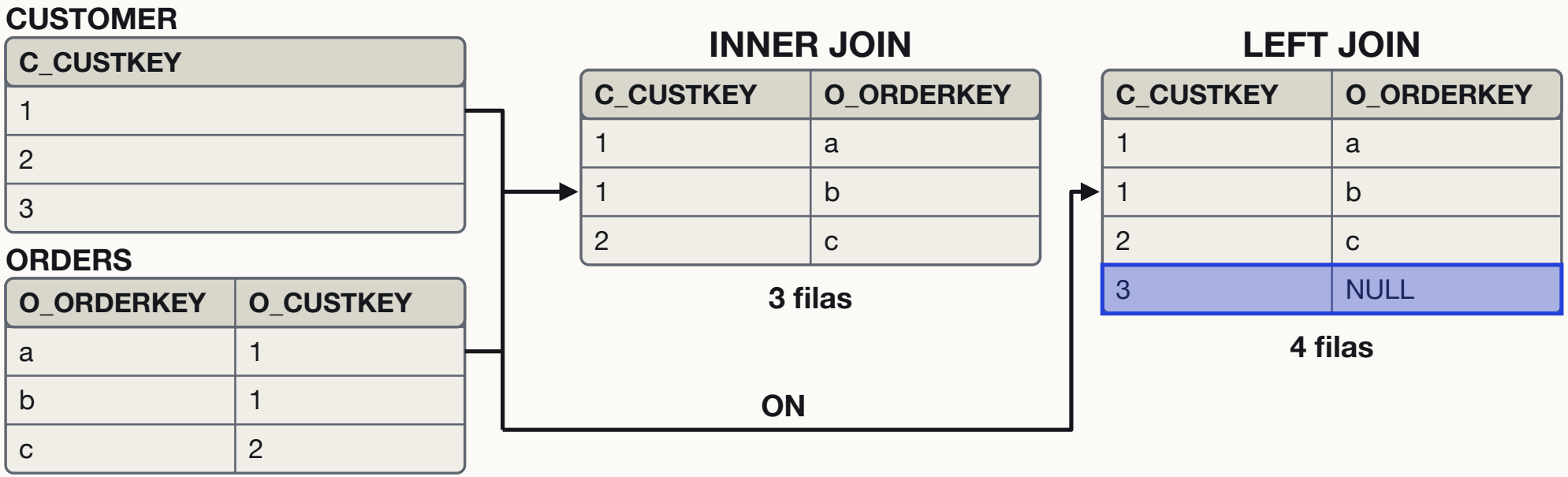
05

LAB 2

¿Qué le pasa a un total cuando unes tablas de distinta cardinalidad (cuántas filas de una corresponden a una de la otra)?

JOIN combina filas de dos tablas por una condición: INNER conserva solo las parejas; LEFT, todas las de la izquierda

JOIN combina filas cuando se cumple la condición ON: C_CUSTKEY = O_CUSTKEY. Ejemplo ilustrativo.



En SF1, **50 004** de 150 000 clientes no tienen pedidos: un INNER JOIN los pierde, un LEFT JOIN los conserva.

En SF1, 50 004 clientes no tienen pedidos.

Unir pedidos con líneas repite cada pedido: 1.5 M de filas pasan a 6.0 M y la suma se infla 5 veces

Un pedido (uno) tiene varias líneas (muchos): al unir, el pedido se repite por cada línea.

ORDERS
pedido A · O_TOTALPRICE 100

LINEITEM
A-1
A-2
A-3

JOIN →

Resultado del JOIN
A-1 · **100**
A-2 · **100**
A-3 · **100**

SUM(O_TOTALPRICE) = 300; el pedido vale 100

1 500 000 → 6 001 215

filas: pedidos contra filas tras el JOIN (× 4.0)

226.8 → 1 134.4 mil millones

SUM(O_TOTALPRICE) sin JOIN contra con JOIN (× 5.0)

El monto crece más que las filas porque los pedidos grandes tienen más líneas y se repiten más.

SUM(O_TOTALPRICE), SF1: 226.8 contra 1 134.4 mil millones.

Si la clave es única en un lado, el JOIN devuelve una fila por cada fila del otro; si se repite en ambos, multiplica

Unión	Relación	Filas
ORDERS y CUSTOMER	N:1	Una por pedido
ORDERS y LINEITEM	1:N	Una por línea

COUNT(DISTINCT) cuenta cada clave una vez aunque el JOIN la repita; agregar antes de unir llega en la clase 3

```
SELECT C.C_MKTSEGMENT,
       COUNT(*) AS FILAS,
       COUNT(DISTINCT C.C_CUSTKEY) AS CLIENTES
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER C
JOIN SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS O
  ON C.C_CUSTKEY = O.O_CUSTKEY
GROUP BY C.C_MKTSEGMENT;
```

AUTOMOBILE: 297 453 filas y 19 876 clientes. Otra salida: sumar la medida que vive en el lado de las líneas.

Resuelve los seis ★ y verifica: meta 6 de 6

Snowsight

1. Copia lab2.sql con Copy raw file y pégalo.
2. Corre el Paso 0: crea CURSO.LAB2.
3. Resuelve EJ1 a EJ6 (★).
4. EJ7 y EJ8 (★★): duplicación y corrección.
5. Corre lab2_verificacion.sql con Run All.

06

QUERY PROFILE

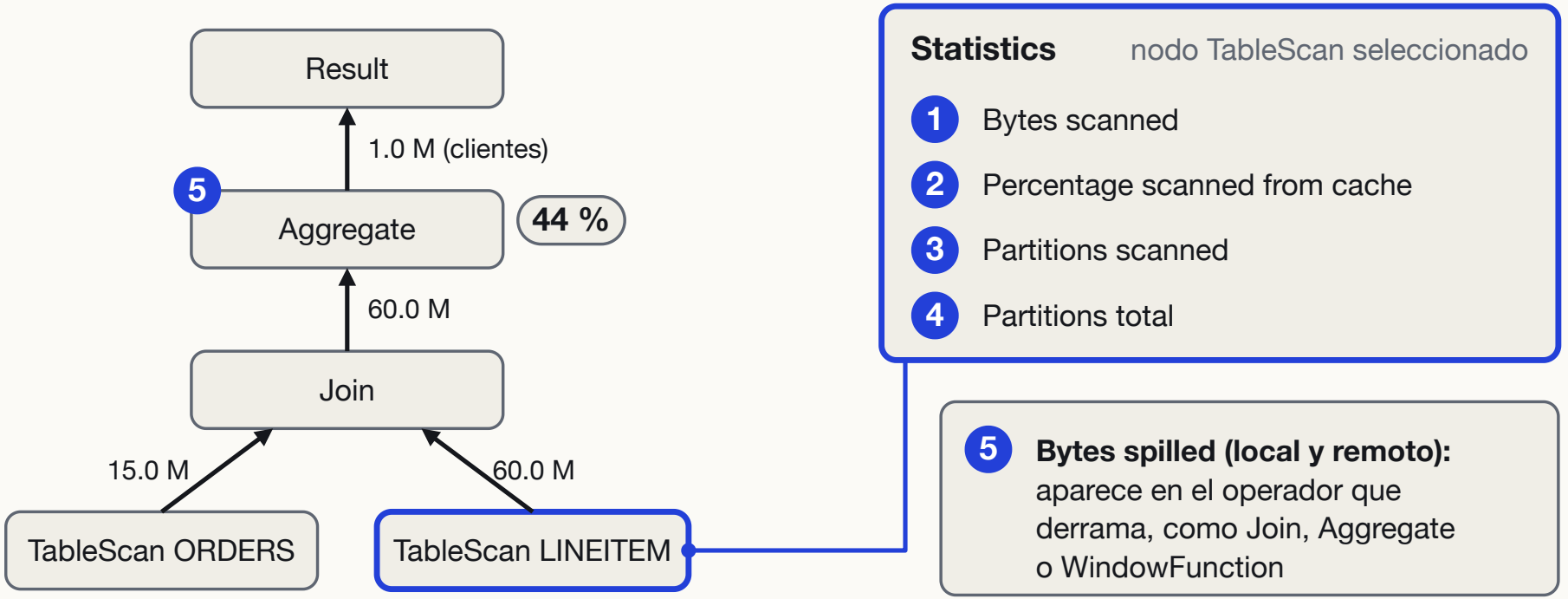
¿Qué evidencia deja cada consulta y qué decisión permite tomar?

El perfil sale de la consulta ejecutada

Snowsight

1. `ALTER SESSION SET USE_CACHED_RESULT = FALSE;`
2. Resultados: Query Details > botón de tres puntos > View Query Profile.
3. O Monitoring > Query History > consulta > pestaña Query Profile.
4. SQL: `SELECT * FROM TABLE(GET_QUERY_OPERATOR_STATS(LAST_QUERY_ID()));`

El perfil es un árbol de operadores: los datos suben desde TableScan y cada nodo muestra su tiempo y sus filas



> Se lee de abajo hacia arriba: filas que suben, tiempo por nodo y, en el nodo elegido, lectura y derrame.

Aggregate agrupa; Sort ordena; WindowFunction calcula sobre ventanas de filas.

Cada síntoma del perfil apunta a una causa y a una decisión distinta

Síntoma	Causa	Decisión
Particiones leídas = totales	Filtro sin pruning	**Filtrar por columna ordenada**
Muchos bytes en TableScan	Columnas de más	**Proyectar solo las usadas**
Join entrega más filas que su entrada mayor	Llave incompleta	**Revisar grano y llaves**
Join o Aggregate dominan	Muchas filas antes de unir	**Agregar antes de unir**
Bytes spilled	Falta memoria	**Menos datos o más talla**
Sort domina sin necesidad	ORDER BY intermedio	**Quitarlo**

Las cuatro slides siguientes miden cuatro de estas filas.

Proyectar 3 columnas lee 4 veces menos y tarda 5 veces menos, con iguales particiones

LINEITEM SF10, 1995, 2.ª ejecución	SELECT *	3 columnas
Bytes TableScan	1.72 GB	0.45 GB
Particiones	116 de 116	116 de 116
Tiempo	1.46 s	0.29 s

El pruning depende del orden de las filas: YEAR() podó como el rango

Filtro sobre LINEITEM	Escala	Particiones
YEAR(L_SHIPDATE) o rango 1995	SF100	171 de 1 022
L_QUANTITY = 10	SF100	1 022 de 1 022
Rango 1995	SF10	116 de 116

Unir por una llave incompleta multiplica las filas por 4 sin error: el Join lo delata

LINEITEM con PARTSUPP, SF10	Solo PARTKEY	PARTKEY y SUPPKEY
Filas	239.9 M	60.0 M
Peso del Join	89 %	63 %
Tiempo	1.42 s	0.87 s

Si el perfil muestra spilling, subir a Medium lo hizo 4.4 veces más rápido, casi al mismo costo

Ventana sobre 600 M filas	X-Small	Medium
Bytes spilled local (a disco)	4.73 GB	0
Tiempo	48.4 s	11.0 s
Créditos	0.0134	0.0122

MINI-CASO

¿Qué le contestas a la dirección comercial del mayorista, y cuánto cuesta leer cada respuesta?

TPC-H SF10 · cuatro consultas · un Query Profile

Cuatro preguntas de negocio, cuatro vistas con columnas fijas

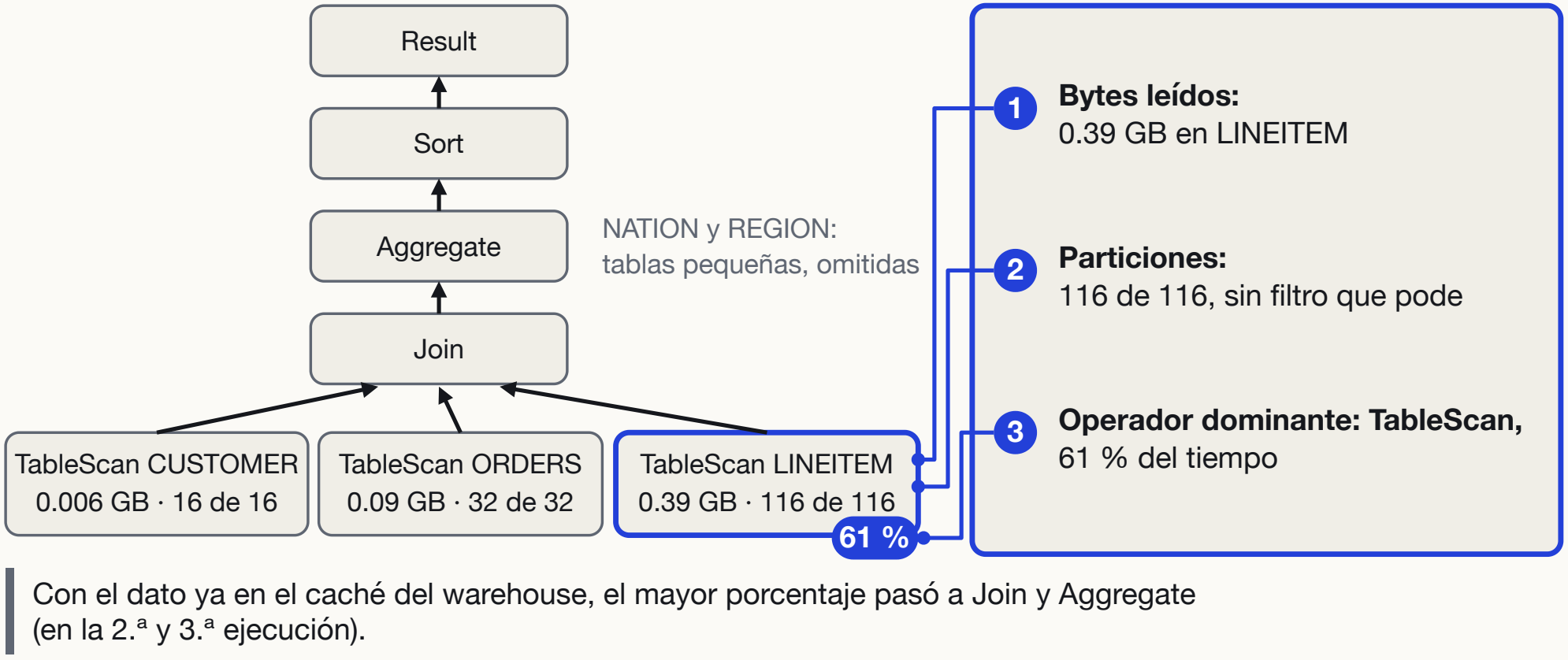
Vista	Pregunta
Q1	¿Cuánto vendió cada región en 1995?
Q2	¿Qué naciones superan 600 000 pedidos?
Q3	¿Cuántos clientes por región nunca compraron?
Q4	¿Qué modo de envío mueve más importe por región?

Escribe Q1 y Q4 en CURSO.CASO y verifica antes del perfil

Snowsight

1. Copia caso.sql y corre su Paso 0.
2. Escribe Q1 y Q4; Q2 y Q3, después de clase.
3. Corre caso_verificacion.sql con Run All.
4. Con Q4 en OK, abre su perfil.

El Query Profile de la consulta más pesada muestra tres datos: bytes leídos, particiones y el operador que más tarda



> Con el dato ya en el caché del warehouse, el mayor porcentaje pasó a Join y Aggregate (en la 2.ª y 3.ª ejecución).

Q4 en SF10, X-Small, primera ejecución.

Q1 filtra un año y no poda en SF10; Q4 une cinco tablas y lee todo lo que une

Consulta	Tabla mayor	Bytes	Particiones
Q1	ORDERS	0.11 GB	32 de 32
Q4	LINEITEM	0.39 GB	116 de 116

DECISIÓN

¿Qué cambiarías en Q4 para que lea menos, y qué no cambiaría el tamaño del warehouse?

Bytes, particiones y operador dominante

Leer big data rápido es leer menos: columnas, particiones y caché

Problema	Síntoma con dato	Idea de ingeniería	Dónde sigue
1 **Columnar**	0.60 GB frente a 6.37 GB	Leer solo columnas	**Hoy: Lab 1, Lab 2 y mini-caso**
2 **Pruning**	134 de 10 336 particiones	Omitir por rango	**Hoy: Lab 1, Lab 2 y mini-caso**
3 **Escalar**	10× filas, unas 9× tiempo	Separar cómputo y datos	**Hoy: demos**
4 **Aislar cargas**	Carga de 6.0 s con consulta de 0.79 s	Cómputo por carga	**Hoy: demos**
5 **Costo y elasticidad**	Medium: 1.45× más rápido, 2.8× el costo	Pagar lo encendido	**Hoy: demos**
6 **Datos crudos**	15 de 24 columnas usadas	Esquema flexible	Clase 4: lake en S3
7 **Transformación**	Filtros, 2 uniones, aplanado	Código con pruebas	Clases 5 y 6: Spark y Airflow
8 **Formatos abiertos**	5.4 GB de Parquet en S3	Archivos abiertos	Clase 4: Parquet en S3

Los 239.5 M viajes NYC se leerán con estos principios (columnas, particiones por fecha) en las clases 4 a 6.

La clase 3 agrega subconsultas, CTE y funciones de ventana, y abre el modelado dimensional

Hoy	Clase 3
JOIN que duplica	Agregar antes de unir
GROUP BY y HAVING	Subconsultas y CTE
Un total por grupo	Funciones de ventana
TPC-H	Modelo dimensional

Con las cuotas aprobadas, lanza el bootstrap desde tu fork (paso 4.1)

GitHub

1. Abre la pestaña Actions de tu fork.
2. Elige «Enlace de bootstrap» y pulsa Run workflow.
3. Abre el run que lanzaste.
4. Pulsa Launch Stack: se abre la consola de AWS.

En la consola de AWS crea el stack y espera CREATE_COMPLETE (paso 4.2)

Consola AWS

1. VpcId: la VPC default. SubnetIds: todas sus subnets.
2. AlertEmail: tu correo.
3. Marca la casilla IAM y pulsa Create stack.
4. Espera 3 a 4 min: bigdata-bootstrap en CREATE_COMPLETE.